

PROJECT TIMELINE

Indirect Prompt Injection Propagation in Agentic AI Pipelines

Target: UTRGV STEM Research Conference 2026 | Duration: 4 Weeks | Deadline: April 24, 2026

TIMELINE SUMMARY

Week	Focus	Key Deliverable	Completion Gate	Hours
1	Core Infrastructure + RAG Pipeline	Working RAG pipeline with adversarial doc retrieved and logged	Query returns altered output when adversarial doc is at rank 1	12–14 hrs
2	Linear Chain + Parallel Pipelines	All three pipeline topologies operational with per-hop logging	All 3 agent outputs + aggregated output logged in a single run	12–14 hrs
3	Metrics + Full Experiments	All 9 experiment runs complete; taxonomy table fully populated	All 3 metrics return non-null values; CSV exported successfully	12–14 hrs
4	Poster Design + Rehearsal	Print-ready poster file + full rehearsed walkthrough	Poster submitted to library print queue by Day 26	10–12 hrs

WEEK 1 — Core Infrastructure + RAG Pipeline | Hours Budget: ~12–14 hrs

Goals

- **Goal 1:** Stand up corpus_loader.py, embedding pipeline, and FAISS index against the 4 benign + 1 adversarial doc corpus
- **Goal 2:** Implement llm_client.py wrapping Ollama (LLaMA 3.1 8B) with Groq fallback via GROQ_FALLBACK=1
- **Goal 3:** Build Notebook 1: LangChain RetrievalQA pipeline with configurable retrieval k
- **Goal 4:** Log the full assembled prompt verbatim before and after each LLM call
- **Goal 5:** Confirm adversarial doc is retrievable at rank 1 for at least one test query

Tasks by Day-Range

Days	Task	Detail
Days 1–2	Environment + Corpus	Install dependencies (LangChain, FAISS, sentence-transformers, Ollama). Write corpus_loader.py to tag benign vs. adversarial docs. Build FAISS index. Confirm Ollama is serving LLaMA 3.1 8B locally with a smoke-test query. Set up Groq API key and test GROQ_FALLBACK=1.
Days 3–4	LLM Wrapper + Retriever	Implement llm_client.py with a stateless call interface (prompt, system_prompt, max_tokens → text). Wire the FAISS retriever to return top-k chunks with similarity scores. Verify the adversarial document is retrieved at rank 1 using the chosen test query Q.
Days 5–7	RAG Pipeline Notebook	Build Notebook 1 using LangChain RetrievalQA. Log full assembled prompt (system message + chunks + Q) verbatim as a pre-generation entry. Log the generated response as a post-generation entry. Run Baseline and a manual Injected-Rank-1 trial to confirm altered output is observable.

☑ **COMPLETION GATE:** By end of Week 1, I must be able to run the RAG pipeline and show that the adversarial document at rank 1 produces a final output that differs observably from the Baseline — with both pre- and post-generation log entries present in the .jsonl file.

⚠ **RISK FLAG:** Ollama inference speed on local hardware. LLaMA 3.1 8B at 4-bit quantization can be slow (15–45 sec/query) depending on GPU availability. If latency blocks iterative testing, activate Groq fallback immediately. Do not spend more than 2 hours debugging Ollama performance before switching — log the issue and move on.

WEEK 2 — Linear Chain + Parallel Pipelines | Hours Budget: ~12–14 hrs

Goals

- **Goal 1:** Build Notebook 2: 3-node LangChain LLMChain with Agent 1 (summarizer), Agent 2 (synthesizer), Agent 3 (formatter)
- **Goal 2:** Implement per-hop output logging — each agent output stored before passing to the next node
- **Goal 3:** Build Notebook 3: AutoGen GroupChat with 3 parallel agents + aggregator node
- **Goal 4:** Apply max_turns=5 hard cap and round_robin speaker selection; log aggregator output as a separate entry
- **Goal 5:** Run a manual smoke test of all three topologies end-to-end with a Baseline query

Tasks by Day-Range

Days	Task	Detail
Days 8–9	Linear Chain Build	Implement Notebook 2 using LangChain LLMChain. Assign roles: Agent 1 receives user query Q + corpus chunks; Agents 2 and 3 receive only the prior agent's output. Log each agent's raw output immediately after generation. No agent after Agent 1 touches the corpus directly.
Days 10–11	Linear Chain Validation	Run the full linear chain with Baseline corpus (no adversarial doc). Confirm all three agent outputs are independently logged. Run one Injected-Rank-1 trial and verify that the injection artifact is present in at least Agent 1's output. Spot-check that log file rotates correctly per run.
Days 12–14	Parallel Pipeline Build + Smoke Test	Implement Notebook 3 using AutoGen GroupChat. Configure 3 agents + aggregator. Apply max_turns=5. Log all three parallel agent outputs as separate entries; log aggregator output as a fourth. Run Baseline smoke test across all three topologies. Fix any non-termination issues in AutoGen before closing the week.

☑ **COMPLETION GATE:** By end of Week 2, I must be able to run all three pipeline topologies — RAG, Linear Chain, and Parallel — on a Baseline query and produce a .jsonl log file containing: (RAG) 2 entries, (Linear) 4 entries, (Parallel) 4 entries, with no null outputs.

⚠ **RISK FLAG:** AutoGen GroupChat non-termination. AutoGen agents can loop indefinitely without a hard max_turns cap and reliable termination conditions. If AutoGen behaves unexpectedly, fall back to a manual orchestration loop using direct llm_client.py calls — this sacrifices framework authenticity but preserves the parallel topology measurement. Flag it as a known limitation in the taxonomy. Do not allow AutoGen instability to consume more than 3 hours before pivoting.

WEEK 3 — Metrics Module + Full Experiment Runs | Hours Budget: ~12–14 hrs

Goals

- **Goal 1:** Implement all three metric functions: `propagation_depth()`, `integrity_score()`, `compromise_signal()`
- **Goal 2:** Calibrate the `integrity_score` threshold (target: 0.85) using pilot runs from Weeks 1–2
- **Goal 3:** Execute all 9 experiment runs (3 topologies × 3 corpus configs: Baseline, Injected-Rank-1, Injected-Rank-3)
- **Goal 4:** Populate the taxonomy table with empirical quantitative values replacing all qualitative placeholders
- **Goal 5:** Export final taxonomy to CSV and produce 2–3 supporting charts (`compromise_signal` rate, `propagation_depth` by topology)

Tasks by Day-Range

Days	Task	Detail
Days 15–16	Metrics Module	Implement <code>metrics.py</code> with <code>propagation_depth()</code> (hop-order log comparison), <code>integrity_score()</code> (<code>difflib.SequenceMatcher</code> ratio, per-hop and final), and <code>compromise_signal()</code> (Stage 1: regex match on artifact string; Stage 2: <code>integrity_score</code> < 0.85 + instruction-following check). All metrics computed from persisted logs only — never from live pipeline state.
Days 17–18	Calibration + Pilot Runs	Use existing Baseline and Injected-Rank-1 log files from Weeks 1–2 to calibrate the 0.85 threshold. Confirm Baseline runs return: <code>propagation_depth=0</code> , <code>integrity_score=1.0</code> , <code>compromise_signal=False</code> . Confirm at least one Injected-Rank-1 run returns <code>compromise_signal=True</code> . Document threshold decision in the experiment notebook.
Days 19–21	Full 9-Run Experiment + Taxonomy	Execute all 9 runs in sequence (per-topology: Baseline → Injected-Rank-1 → Injected-Rank-3). Record all metric values. Populate the taxonomy table replacing qualitative risk labels with quantitative averages. Export to CSV via <code>pandas</code> . Generate bar charts of <code>compromise_signal</code> rate and <code>propagation_depth</code> by topology using <code>matplotlib</code> or <code>seaborn</code> . Save all chart files.

☑ **COMPLETION GATE:** By end of Week 3, I must be able to open the exported CSV and show that all 9 runs have non-null values for all 3 metrics, and that the taxonomy table contains quantitative values in the Exploitation Risk and Defender Visibility columns — with at least one Injected-Rank-1 run showing `compromise_signal=True`.

⚠ **RISK FLAG:** Paraphrase evasion false negatives: the LLM may execute the injected instruction but rephrase it, causing Stage 1 of `compromise_signal()` to miss. If this occurs, rely on the Stage 2 behavioral check (`integrity_score` delta). Document any paraphrase evasion cases as edge-case findings — they strengthen the taxonomy rather than invalidating it. Also flag: if Injected-Rank-3 produces no detectable injection across all topologies, report this as a meaningful null result (retrieval rank as a natural gate) rather than a failure.

WEEK 4 — Poster Design, Polish, and Rehearsal | Hours Budget: ~10–12 hrs

Goals

- **Goal 1:** Design and complete the 36" × 48" portrait poster using the finalized taxonomy table and charts from Week 3
- **Goal 2:** Submit the poster file to the UTRGV Library print queue no later than Day 26 to allow printing time

- **Goal 3:** Conduct at least two full rehearsed walkthroughs (poster format: ~5-min explanation to a non-CS STEM audience)
- **Goal 4:** Prepare 3–5 anticipated Q&A responses covering: What is prompt injection? Why does topology matter? What are the real-world implications?
- **Goal 5:** Final review of all content for accuracy; confirm no empirical claims exceed what the experiment data supports

Tasks by Day-Range

Days	Task	Detail
Days 22–23	Poster Layout + Content	Create the poster in PowerPoint, Canva, or Adobe Illustrator at 36" × 48" portrait. Sections: Title/Author block, Motivation (trust-violation hook from abstract), System Architecture diagram, Experiment Protocol, Taxonomy Table, Key Findings (2–3 highlighted results), Implications for Practitioners, Acknowledgements. Use UTRGV branding colors.
Days 24–26	Polish + Print Submission	Finalize all charts and figures at print resolution (300 DPI minimum). Confirm taxonomy table is readable from 10 feet — increase font size if needed. Check all text is clear and jargon is defined. Export final poster as PDF. Submit to University Library print queue by Day 26. Keep a local backup copy of the PDF.
Days 27–28	Rehearsal + Q&A Prep	Conduct two full rehearsed poster walkthroughs with a timer (target: 4–5 minutes). Practice the non-technical opening: lead with the trust-violation hook. Prepare written notes for Q&A: What is prompt injection? How does topology change risk? What would a defender do with this taxonomy? Run a final accuracy check — verify every number on the poster matches the exported CSV.

☒ **COMPLETION GATE:** By end of Week 4 (Day 26 specifically), the poster PDF must be submitted to the library print queue. By Day 28, I must be able to deliver a 4–5 minute explanation of the research to a non-CS audience from memory, with the taxonomy table as the anchor.

SLIP RECOVERY PROTOCOL | Minimum Viable Scope if a Week Falls Behind

If Week 1 slips:

- **Cut:** Skip implementing Groq fallback initially — use only Ollama. Add the fallback in Week 2 if needed.
- **Cut:** Use a pre-built FAISS index from a single test run rather than rebuilding per experiment. Implement proper rotation in Week 2.
- **Must keep:** A working RAG pipeline with at least one logged end-to-end run showing adversarial doc retrieved.

If Week 2 slips:

- **Cut:** If AutoGen is non-terminating after 3 hours of debugging, replace the Parallel pipeline with a manual orchestration loop (3 × direct llm_client.py calls + a simple aggregator function). Note as a known limitation.
- **Cut:** Reduce Linear Chain from 3 nodes to 2 nodes (summarizer → formatter) and note scope reduction.
- **Must keep:** At least 2 of 3 pipeline topologies fully operational before Week 3 begins.

If Week 3 slips:

- **Cut:** Reduce from 9 runs (3 × 3) to 6 runs (3 topologies × Baseline + Injected-Rank-1 only). Drop Injected-Rank-3.
- **Cut:** Drop chart generation — present the taxonomy table as the sole visual output.
- **Must keep:** The taxonomy table must have at least one empirical row per topology. If a topology produced zero signal, report it as a null result with a brief explanation.

If Week 4 slips:

- **Cut:** Simplify poster layout to a single-column format if design time is short. Readability over aesthetics.
- **Hard deadline:** Do not miss the library print submission. A simple readable poster printed on time is better than a polished poster that misses printing.

WHAT NOT TO BUILD | Explicit Scope Boundaries for This Submission

The following items are explicitly out of scope for the April 24 conference submission. They are documented here to prevent scope creep during implementation.

Category	Out-of-Scope Item
Defender Layer	Pre-screening agent that inspects retrieved documents before pipeline execution. Deferred to v2.
Injection Variants	Markup injection, whitespace injection, Unicode injection. Phase 1 uses body-text injection only.
Model Comparison	Running experiments against a second LLM (Mistral 7B, Phi-3 Mini). Single model only for this submission.
Trust Scoring	Configurable trust_score metadata field on corpus documents and trust-gated retrieval. Deferred to v2.
SQLite Migration	Replacing .jsonl log files with a SQLite database. JSON lines is sufficient for 9 experiment runs.
Fourth Topology	Hub-and-spoke topology (orchestrator + specialist sub-agents). Three topologies only for this submission.
Adversarial Gen.	Parametric adversarial document generator varying injection placement and phrasing. Manual doc authoring only.
Metadata Injection	Adversarial instruction embedded in document metadata fields. Body-text injection only in Phase 1.
CrewAI	CrewAI framework excluded from implementation. No demonstrations have been run; cannot make empirical claims.
Production Deployment	No cloud hosting, API endpoints, or production-grade infrastructure. Local environment only.